

✓ Calcoliamo delle aree: metodo dei rettangoli

L'idea del metodo dei rettangoli (detto anche approssimazione secondo Riemann) per approssimare le aree è la seguente:

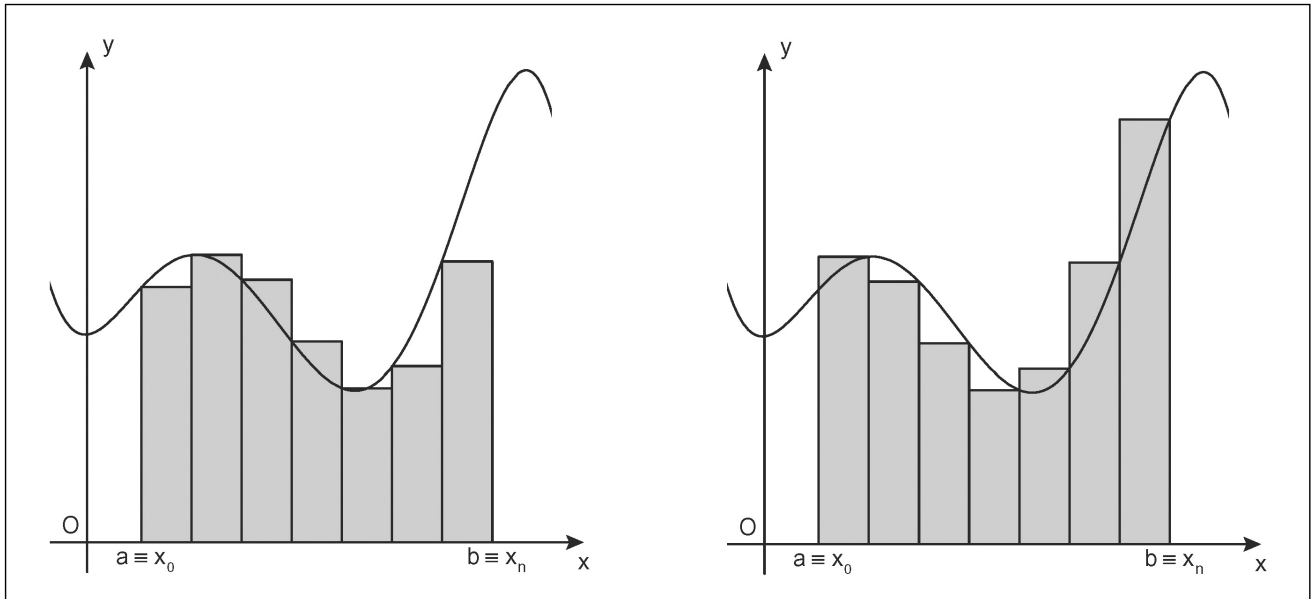
$$\int_a^b f(x)dx \approx \sum_{i=0}^{n-1} h f(x_i)$$

con h ampiezza dell'intervallo.

Oppure:

$$\int_a^b f(x)dx \approx \sum_{i=1}^n h f(x_i)$$

Qui di seguito inseriamo una immagine esemplificativa del metodo.



Nelle successive celle di codice inseriamo una implementazione possibile applicata al calcolo di: $A = \int_0^2 x^2 dx = [x^3/3]_0^2 = 8/3$

```
# codice che spiega la suddivisione degli intervalli e i valori delle x prese
# usando opzione x[:n-1] e x[1:] (ricordando che python parte da 0)
import numpy as np

a = 0
b = 2
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)

print("Valore h: ", h)
print("Vettore x: ", x)

print(x[:n-1]) # estremi sinistri degli intervalli

print(x[1:]) # estremi destri degli intervalli
```

```
Valore h: 0.2
Vettore x: [0.  0.2 0.4 0.6 0.8 1.  1.2 1.4 1.6 1.8 2. ]
[0.  0.2 0.4 0.6 0.8 1.  1.2 1.4 1.6 1.8]
[0.2 0.4 0.6 0.8 1.  1.2 1.4 1.6 1.8 2. ]
```

```
a = 0
b = 2
n = 101
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)

f = x**2 # calcolo x^2

I_riemannL = h * sum(f[:n-1])
err_riemannL = 8/3 - I_riemannL

I_riemannR = h * sum(f[1:])
```

```
err_riemannR = 8/3 - I_riemannR
```

```
print(8/3)
print(I_riemannL)
print(err_riemannL)
```

```
print("\n")
print(8/3)
print(I_riemannR)
print(err_riemannR)
```

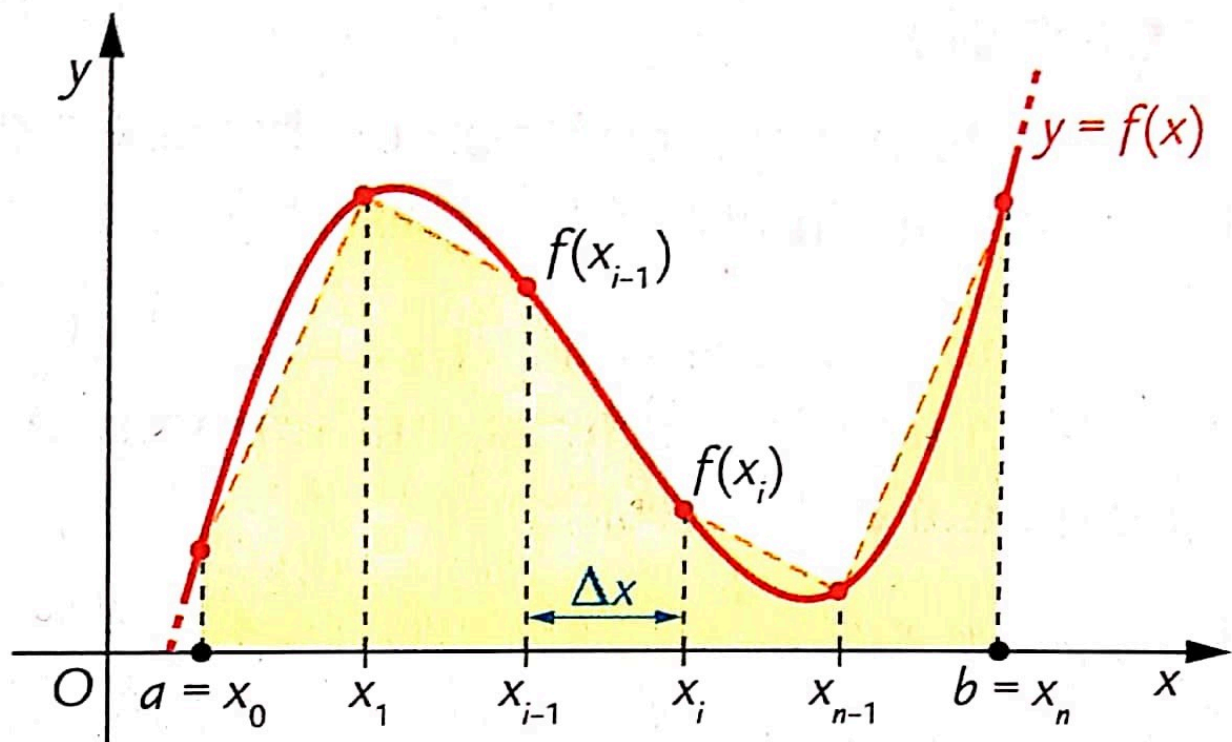
```
2.6666666666666665
2.6268000000000002
0.03986666666666627
```

```
2.6666666666666665
2.7068000000000003
-0.0401333333333338
```

✓ Calcoliamo delle aree: metodo dei trapezi

L'idea del metodo dei trapezi

$$\int_a^b f(x)dx \approx \frac{\Delta x}{2} \sum_{i=0}^{n-1} (f(x_i) + f(x_{i+1}))$$



Si osserva in realtà che il metodo dei trapezi definito in precedenza "conta due volte" molti dei termini.

Un modo più computazionalmente efficiente per calcolarlo è il seguente:

$$\int_a^b f(x)dx \approx \frac{\Delta x}{2} \left(f(x_0) + 2 \left(\sum_{i=1}^{n-1} f(x_i) \right) + f(x_n) \right)$$

```
# codice per capire meglio codice dopo
import numpy as np

# voglio dividere l'intervallo che va da 1 a 10 in 9 intervallini
a=1
b=10
N=9
x = np.linspace(a,b,N+1)
print("x= ",x)

y=x*x # calcolo la funzione x^2 nel vettore x
print("y= ",y)
```

```
y_right= y[1:]
print("y_right= ",y_right)
```

```
y_left = y[:-1]
print("y_left= ", y_left)
```

```
x= [ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
y= [ 1.  4.  9. 16. 25. 36. 49. 64. 81. 100.]
y_right= [ 4.  9. 16. 25. 36. 49. 64. 81. 100.]
y_left= [ 1.  4.  9. 16. 25. 36. 49. 64. 81.]
```

```
def metodo_trapezi(f,a,b,N=50):
    """metodo dei trapezi con:
    - f funzione
    - a,b estremi integrale
    - N numero di punti
    """
    x = np.linspace(a,b,N+1) # N+1 punti per avere n "intervallini"
    y = f(x)
    y_right = y[1:] # right endpoints
    y_left = y[:-1] # left endpoints
    dx = (b - a)/N
    T = (dx/2) * np.sum(y_right + y_left) # calcolo area trapezio
    return T
```

Esempio:

$$A = \int_0^2 x^2 dx = [x^3/3]_0^2 = 8/3$$

```
f = lambda x: x**2

print("Test con N= 50")
res= metodo_trapezi(f,0,2,N=50)
print(res)
print(8/3)
print("Errore: ", res-8/3)

print("\n")
print("test con N=100")
res= metodo_trapezi(f,0,2,N=100)
print(res)
print(8/3)
print("Errore: ", res-8/3)
```

```
Test con N= 50
2.6672000000000002
2.6666666666666665
Errore: 0.00053333333333337187
```

```
test con N=100
2.6667999999999994
2.6666666666666665
Errore: 0.0001333333333333287456
```

```
import numpy as np
import math
from scipy.stats import norm # Usiamo norm per il calcolo della PDF standard

def gaussian_pdf(x, mu=0, sigma=1):
    """
    Funzione di densità di probabilità della Curva Gaussiana.
    mu (media) e sigma (deviazione standard) sono i parametri standard.
    """
    # Usiamo la formula standard
    coefficient = 1 / (sigma * math.sqrt(2 * math.pi))
    exponent = -0.5 * ((x - mu) / sigma)**2
    return coefficient * np.exp(exponent)

def trapezoid_area(f, a, b, n, *args, **kwargs):
    """
    Approssima l'area sottesa dalla funzione f(x) tra a e b
    usando il metodo del trapezio con n suddivisioni.

    Parametri:
    f (function): La funzione da integrare (qui, gaussian_pdf).
    a (float): Limite inferiore.
```

```

b (float): Limite superiore.
n (int): Numero di suddivisioni (più alto = più preciso).
*args, **kwargs: Parametri della funzione f (es. mu, sigma).
"""
if n <= 0:
    raise ValueError("Il numero di suddivisioni (n) deve essere positivo.")

# 1. Calcola la larghezza di ogni intervallo (h)
h = (b - a) / n

# 2. Genera i punti x (inclusi a e b)
x = np.linspace(a, b, n + 1)

# 3. Calcola i valori della funzione f(x) in questi punti
y = f(x, *args, **kwargs)

# 4. Applica la regola del trapezio:
# Area = h/2 * [y_0 + 2*y_1 + 2*y_2 + ... + 2*y_{n-1} + y_n]

# Somma gli elementi centrali (2*y_i)
area_somma = np.sum(y[1:-1]) * 2

# Aggiunge gli estremi (y_0 e y_n)
area_somma += y[0] + y[-1]

# Moltiplica per h/2
area_approssimata = (h / 2) * area_somma

return area_approssimata

# --- Esempio di Utilizzo ---

# Parametri della Gaussiana Standard (Distribuzione Normale Standard)
MU = 0 # Media
SIGMA = 1 # Deviazione Standard

# Intervallo di integrazione (es.  $P(0 < X < 1)$ )
a = 0.0
b = 1.0

# Numero di intervalli per l'approssimazione
N_TRAPEZOIDS = 1000

# Calcola l'area approssimata
area_approssimata = trapezoid_area(gaussian_pdf, a, b, N_TRAPEZOIDS, mu=MU, sigma=SIGMA)

print(f"--- Area sotto la Curva Gaussiana ---")
print(f"Parametri: Media ( $\mu$ )={MU}, Dev. Std. ( $\sigma$ )={SIGMA}")
print(f"Intervallo di integrazione: [{a}, {b}]")
print(f"Suddivisioni (n): {N_TRAPEZOIDS}")
print(f"Area approssimata (Probabilità): {area_approssimata:.8f}")

# Valore esatto (utilizzando la funzione CDF di SciPy come verifica)
# La CDF (Cumulative Distribution Function) calcola l'area esatta  $P(X < b) - P(X < a)$ 
area_esatta = norm.cdf(b, loc=MU, scale=SIGMA) - norm.cdf(a, loc=MU, scale=SIGMA)
print(f"Area esatta (verifica): {area_esatta:.8f}")

--- Area sotto la Curva Gaussiana ---
Parametri: Media ( $\mu$ )=0, Dev. Std. ( $\sigma$ )=1
Intervallo di integrazione: [0.0, 1.0]
Suddivisioni (n): 1000
Area approssimata (Probabilità): 0.34134473
Area esatta (verifica): 0.34134475

```

